

Lab 3 - Connecting Language to fMRI reads, Stat 214, Spring 2026

These are the **final project specific** instructions. Please also see the general lab instructions in **lab-instructions.pdf**.

Contents

1	Submission	2
	Subpart Check-Ins	2
	Final Report	2
	Special coding instructions	2
2	Academic honesty and teamwork	2
	Academic honesty statement	2
	Collaboration policy	3
	LLM usage policy	3
3	Lab 3 Overview	3
	Data	4
	Coding	4
4	Instructions	4
	Data Preprocessing	4
	Lab 3.1	4
	Part 1: Generate Embeddings	4
	Part 2: Modeling & Evaluation	5
	Lab 3.2	6
	Part 1: Pre-trained and Fine-tuned BERT	6
	Part 2: Modeling & Evaluation	6
	Lab 3.3	7
	Part 1: Interpretation	7
	Part 2: Final Report	7
5	Note on Grading	7

1 Submission

This lab is broken up into three sub-parts, where each sub-part will have a check-in due date. The final lab report will include your results from all three sub-parts. For the final submission, push your final code to your **stat-214** GitHub repository by **23:59 on Friday May 8th**. Your push access will be cutoff promptly at midnight, so take care not to be late as late labs will not be accepted.

Subpart Check-Ins

There will be two sub-part check-ins due before the final report:

3.1: due April 10th, 23:59

3.2: due April 24th, 23:59

Each check-in will have a bcourses assignment. Please have everyone in your group submit the same check-in pdf. Each check-in submission should be 1-2 pages and include a summary of your results for that sub-part, including at least one figure.

Final Report

The page limit for this lab is 20 pages. The bibliography and academic honesty sections don't count towards this limit.

Follow the general lab instructions in [stat-214-gsi/discussion/week1/lab-instructions.pdf](#) for more details. Please do not try to make your project fit the requirements at the last minute!

I have not provided a template as a guideline for the writeup. You should have clean code that would be clear to reproduce and a report structure that is compelling to read and roughly follows the conceptual progression of previous labs. The final report will should be in your repository by **23:59 on Friday May 8th**.

Special coding instructions

For any ridge regression model you train, please save the trained model in your results directory. It should be a .pkl file.

2 Academic honesty and teamwork

Academic honesty statement

You can use your statement from lab1 or lab2.

Collaboration policy

Within-group: In a file named `collaboration.txt` in your `report` directory, you must include a statement detailing the contributions of each student, which indicates who worked on what. After the labs are submitted, I will also ask you to privately review your group members' contributions.

With other people: You are welcome to discuss ideas with the GSIs. Please avoid discussing specific ideas with other groups, as I don't want to receive 5 of the same lab. If you do consult with students in other groups, you must acknowledge these students in your lab report.

LLM usage policy

You are allowed to use LLMs (ChatGPT, GitHub Copilot, etc.) to *assist* in this lab, but are not allowed to use it for more than creating visualizations or helping correct grammar in the report. If we have reason to believe an LLM wrote a significant portion of your code (more than 5%) without your editing or iteration, or any section of your report word-for-word, this will constitute an honor code violation.

3 Lab 3 Overview

Goal The final project focuses on predicting blood-oxygen level across brain voxels 1 via an fMRI as subjects listen to various podcasts. Specifically, the goal is to create embeddings from the text of the podcasts to predict voxels. Overall your tasks will include:

- Using NLP techniques to extract text embeddings including using/fine-tuning pre-trained large scale LLMs.
- Training a linear model to predict voxels from the text embeddings
- Interpreting results via different techniques

Scientific Motivation A key aspect of intelligence is the ability of our brains to process rich, complicated language. Accurately predicting how the brain responds to textual stimuli implies that we have a model that could be used to understand how the brain processes language. Further, embeddings from this model can be used to understand representations in the brain. For example, these embeddings can be used to derive insights about how different parts of the brain function to process language. As a result, scientists have dedicated significant effort to measuring brain function via fMRIs. This lab focuses on experiments conducted by Alexander Huth and his lab at UT Austin.

Data

We have data from two subjects listening to stories measured by the Huth Lab at UT Austin [1]. The data for each subject consists of whole-brain blood-oxygen dependent (BOLD) signals measured at various points across the podcast. That is, for each subject-story pair, we have measurements $Y \in \mathbf{R}(T' \times V)$, where T' represents the number of fMRI measurements, and V is the number of voxels. For each subject-story pair, their data can be found on the dropbox link below. Note that these files are really big, $\approx 20\text{GB}$ each. Because of this, the data is already loaded on Bridges2 and can be accessed at the path `"/ocean/projects/mth250011p/shared/215a/final_project/data"`. We also have the raw text for each story on the link above which we will use to generate our embeddings to predict the matrix Y .

Access data at the following link: <https://www.dropbox.com/scl/fo/6taux72r36h9wgv0mza8k/AC04j1WSCT96FIMN2bNtb0s?rlkey=4wpvdb61n3ppl307q77czwl7j&st=jldyyoic&dl=0>

Coding

We provide code to help you process some of the fMRI data. You will find a `ridge utils` folder and under `code/preprocessing.py`, you will find multiple functions that you will use below to process the data. You will also find a very simple code skeleton for BERT Fine-tuning.

4 Instructions

Data Preprocessing

For any of these embeddings, you will find that the embedding dimension is not likely to match the measurement dimensions. Hence you will have to downsample from your embeddings (see [1] for details) and for that you can use the provided **`downsample_word_vectors`** from the **`code/preprocessing.py`** script.

Further, trim the first 5 TRs and last 10 TRs of the output data to better match voxel measurements.

Finally you should use **`make_delayed`** from the **`code/preprocessing.py`** to insert delays from 1 through 4 seconds inclusively - explain what this does and why this might be useful.

Lab 3.1

Part 1: Generate Embeddings

An embedding refers to a numerical vector representation of text. This step is essential to convert our raw text into something a predictive model can use. Before we try embedding, for each subject, split the stories into a training and test set. That is, you will need to fit and evaluate a model *per* subject.

1. Provided the list of stories, generate embedding vectors via bag-of-words. Notice that the dimensions of the dimensions of the resulting matrix do not match up with the response matrix. We will have to down-sample it (see [1] for details) to match dimensions. Explain what is being done here.
2. Call **downsample_word_vectors** from the file `code/preprocessing.py` to get the dimensions to match. Further, trim the first 5 seconds and last 10 seconds of the output.
3. Create lagged versions of the features using **make_delayed** from `code/preprocessing.py` with delays ranging from [1, 4] inclusive. Explain what this does.
4. Repeat the process above for 2 other pre-trained embedding methods: Word2Vec and GloVe. You will have to find these pre-trained embedding methods online, and use them. The processed embeddings resulting from these steps will serve as the features (X matrix) in our regression.
5. Describe potential benefits of using pre-trained embeddings.

Part 2: Modeling & Evaluation

We will now fit a linear model to each of the embeddings you generated in the previous step.

- Fit a ridge regression model, and report the mean correlation coefficient (CC) for different embeddings
- Devise a scheme to cross validate the different models and select the best performing linear regressor. Detail your procedure and report the mean test CC, median test CC, top 1 percentile CC. and top 5 percentile CC.
- For the best embedding, perform a more detailed evaluation by examining the distribution of the CC across voxels. Plot the distribution. What do you notice?
- Does this model perform well across all voxels? What does this imply scientifically? What is a reasonable interpretation criterion for interpreting voxels according to PCS?
- Perform a stability analysis, e.g., examine performance across various test stories, or across subjects.

Lab 3.2

In Lab 3.1, we used bag-of-word and pre-trained embeddings to predict BOLD scores for voxels. In this lab, you will repeat the analysis from part 1 except that you use two different embedding models: first, you will use a pre-trained BERT model, and second, you will use a fine-tuned BERT model. Then, you will fit a ridge regression model to predict voxels just as you did in 3.1.

Part 1: Pre-trained and Fine-tuned BERT

1. Using the following pre-trained BERT model <https://huggingface.co/google-bert/bert-base-uncased>, fit a ridge model on the embeddings as you did in 3.1 to predict across voxels.
2. Next, fine-tune this model for this task.
3. Investigate a parameter-efficient approach to fine-tuning this model such as Low-rank adaptation (LORA). Please refer to the following link as to how to use LORA with Huggingface <https://huggingface.co/docs/diffusers/en/training/lora>
4. Compare the performance of the fine-tuned model to the pre-trained model and the ones you derived in 3.1 and 3.2. Specifically, perform a detailed analysis as in earlier labs to compare the performance of the embeddings across voxels.

Part 2: Modeling & Evaluation

We will now fit a linear model to each of the embeddings you generated in the previous step as you did in 3.1.

- Repeat the analysis you performed in 3.1 and fit a ridge model to predict across voxels.
- Does this model perform well across all voxels? What does this imply scientifically? What is a reasonable interpretation criterion for interpreting voxels according to PCS?
- Compare your results to embeddings you tried in lab 3.1. Perform a detailed analysis to compare the performance of embeddings. For example, do different embedding methods perform equally well across the voxels?

Lab 3.3

The final portion of the lab will focus on interpreting and evaluating your results and writing them up in a comprehensive report.

Part 1: Interpretation

Given the (fine-tuned) BERT model from 3.2, we will now interpret the model by investigating the word importance for various voxels. Specifically, do the following.

- For a given test story, identify the voxels where the model performs well.
- For these voxels, run SHAP (<https://shap.readthedocs.io/en/latest/>) and LIME to (<https://github.com/marcotcr/lime>) to identify influential words that strong affect the response. Why do we only do this for the voxels that we perform well for? Think about the “P” in PCS.
- Compare the words discovered by SHAP and LIME, and visualize them. Do these words make intuitive sense to you? How are they different? How are the words discovered different across voxels?
- Repeat this analysis for another test-story.

Part 2: Final Report

Now it is time to synthesize your analysis into one report. In addition to the standard introduction, methods, results, and conclusions, your report should include:

- The performance over the four different word embedding models: Word2Vec, GloVe, pre-trained BERT, and fine-tuned BERT. Feel free to explore any additional models too.
- The performance of the voxel prediction models across different embedding inputs and different prediction models.
- A section on the interpretability of your fine-tuned BERT model.
- A section on stability of one of your judgment calls (whether that be data cleaning/processing, hyper parameters, models, regularization, visualization choices or other). Thoughtfully and creatively choose a stability study that is not necessarily common place but would make you more confident in your final model in the context of your ultimate goal.

5 Note on Grading

As the final project is a group project, being a good collaborator on the lab will be taken into account for each individual’s grade on the final lab (just as for lab 2). After the lab is submitted, we will send a Google form for each group member to evaluate the collaboration received from other members of their group. The final score for each student will be a combination of the student’s collaboration score from their group mates, and the overall project score.

References

- [1] Shailee Jain and Alexander G. Huth. “Incorporating Context into Language Encoding Models for fMRI”. In: *Advances in Neural Information Processing Systems*. Vol. 31. Curran Associates, Inc., 2018.